

Absolutely. Let's do **Queueable Apex from zero**, in the same code-first way as Future Methods.

The goal is that when you see a question like:

"Create a Queueable class that updates Accounts..."

you should know **exactly what to type, why you're typing it, and how to write the test class.**

Queueable Apex — Code From Zero

First memorize the basic structure:

```
apex

public class MyQueueableClass implements Queueable {

    public void execute(QueueableContext context) {

        // asynchronous code

    }

}
```

And to run it:

```
apex

System.enqueueJob(new MyQueueableClass());
```

That's the basic Queueable pattern.

Now let's understand **every word**.

1. What is Queueable Apex?

Queueable Apex means:

Put a job into Salesforce's asynchronous queue and let Salesforce execute it later.

Think about a queue at a counter:

```
Job 1  
Job 2  
Job 3  
Job 4
```

Salesforce processes those jobs asynchronously.

2. Basic Queueable Syntax

```
apex  
  
public class MyQueueableClass implements Queueable {  
  
    public void execute(QueueableContext context) {  
  
    }  
  
}
```

There are several important pieces.

3. `public`

```
apex  
  
public class MyQueueableClass
```

We're declaring the class as public.

4. `class`

```
apex  
  
public class MyQueueableClass
```

Creates an Apex class.

The class contains our Queueable logic.

5. `implements Queueable`

This is VERY important.

apex

```
public class MyQueueableClass implements Queueable
```

`Queueable` is an Apex interface.

By writing:

apex

```
implements Queueable
```

we're telling Salesforce:

"This class is a Queueable job."

Because we're implementing the Queueable interface, Salesforce expects us to provide the required:

apex

```
execute()
```

method.

6. `execute()`

apex

```
public void execute(QueueableContext context)
```

This is where the actual asynchronous work happens.

Think:

```
System.enqueueJob()
```

↓

```
Queue
```

↓

```
execute()
```

↓

```
Your logic
```

7. QueueableContext

apex

```
QueueableContext context
```

This is the context provided to the Queueable job.

You don't normally need to do anything with it in simple examples.

For example:

apex

```
public void execute(QueueableContext context) {  
  
    // logic  
  
}
```

The variable `context` gives information about the current Queueable execution, including the job's context/ID.

For beginner questions, you can simply write:

apex

```
QueueableContext context
```

and not use `context`.

8. void

apex

```
public void execute(...)
```

The `execute()` method doesn't return anything.

9. How do we actually run Queueable?

This is extremely important:

apex

```
System.enqueueJob(  
    new MyQueueableClass()  
);
```

Let's break it down.

```
new MyQueueableClass()
```

Creates an object of our Queueable class.

```
System.enqueueJob()
```

Tells Salesforce:

"Put this Queueable job into the asynchronous queue."

So:

apex

```
System.enqueueJob(new MyQueueableClass());
```

is basically:

Create the job → put the job in Salesforce's queue.

Example 1 — Simplest Queueable

Question

Create a Queueable Apex class that prints "Hello from Queueable" in the debug log.

Step 1 — Create the class

apex

```
public class MyQueueableClass implements Queueable {  
  
    public void execute(QueueableContext context) {  
  
        System.debug('Hello from Queueable');  
    }  
}
```

```
}
```

That's your Queueable class.

Step 2 — Run it

Execute Anonymous:

apex

```
System.enqueueJob(  
    new MyQueueableClass()  
);
```

Flow:

```
new MyQueueableClass()  
    ↓  
System.enqueueJob()  
    ↓  
Queue  
    ↓  
execute()  
    ↓  
System.debug()
```

10. Testing the Queueable

Now the important part.

apex

```
@isTest  
public class MyQueueableClassTest {  
  
    @isTest  
    static void testQueueable() {  
  
        Test.startTest();  
  
        System.enqueueJob(  
            new MyQueueableClass()  
        );  
  
        Test.stopTest();  
    }  
}
```

```
}  
}
```

Let's understand this **line by line**.

11. @isTest

apex

```
@isTest
```

Tells Salesforce:

This is test code.

12. Test class

apex

```
public class MyQueueableClassTest
```

We're creating a test class.

Usually we name it:

```
MyQueueableClass  
    ↓  
MyQueueableClassTest
```

13. Test method

apex

```
@isTest  
static void testQueueable()
```

This is the actual test method.

14. `Test.startTest()`

apex

```
Test.startTest();
```

Starts the section of the test where we're testing our actual operation.

15. Enqueue the Queueable

apex

```
System.enqueueJob(  
    new MyQueueableClass()  
);
```

This is exactly what we'd use in normal code.

The Queueable is placed into the queue.

16. `Test.stopTest()`

apex

```
Test.stopTest();
```

This is **extremely important** for asynchronous Apex tests.

It tells Salesforce:

Complete asynchronous operations that were started during the test.

So:

apex

```
Test.startTest();  
  
System.enqueueJob(new MyQueueableClass());  
  
Test.stopTest();
```

means:

```
START TEST
  ↓
Put Queueable into queue
  ↓
STOP TEST
  ↓
Queueable executes
```

★ Future vs Queueable syntax

You should already be seeing the difference.

Future

```
apex

@future
public static void myMethod() {

}
```

Called using:

```
apex

MyClass.myMethod();
```

Queueable

```
apex

public class MyClass implements Queueable {

    public void execute(QueueableContext context) {

    }

}
```

Called using:

```
apex

System.enqueueJob(
    new MyClass()
);
```

17. Now let's make it useful

The previous example doesn't actually change Salesforce data.

Let's do a real question.



Example Question 2

Create a Queueable Apex class that accepts an Account Id and changes the Account's Description to "Processed by Queueable". Write a test class for it.

This is exactly the type of question you should practice.

Step 1 — Queueable class

```
apex

public class AccountQueueableProcessor implements Queueable {

    private Id accountId;

    public AccountQueueableProcessor(Id accountId) {

        this.accountId = accountId;

    }

    public void execute(QueueableContext context) {

        Account acc = [
            SELECT Id, Description
            FROM Account
            WHERE Id = :accountId
        ];

        acc.Description = 'Processed by Queueable';

        update acc;

    }

}
```

Now let's understand **EVERY** part.

18. implements Queueable

apex

```
public class AccountQueueableProcessor implements Queueable
```

This makes the class a Queueable job.

Without:

apex

```
implements Queueable
```

Salesforce doesn't treat this class as Queueable.

19. Instance variable

apex

```
private Id accountId;
```

This stores the Account Id.

Why `private` ?

Because we only need this variable inside our Queueable class.

20. Constructor

This is something you didn't need in the basic Future example.

apex

```
public AccountQueueableProcessor(Id accountId) {  
  
    this.accountId = accountId;  
  
}
```

This is a **constructor**.

It runs when we create:

apex

```
new AccountQueueableProcessor(accountId)
```

21. Why do we need a constructor?

We need to pass information into our Queueable.

For example:

apex

```
Id accountId = '001...';
```

Then:

apex

```
new AccountQueueableProcessor(accountId)
```

passes that ID into the constructor.

The constructor receives it:

apex

```
public AccountQueueableProcessor(Id accountId)
```

Then:

apex

```
this.accountId = accountId;
```

stores it in the class variable.

22. What does `this` mean?

This is important.

We have:

apex

```
private Id accountId;
```

and:

apex

```
public AccountQueueableProcessor(Id accountId)
```

Both are called:

```
accountId
```

So:

apex

```
this.accountId = accountId;
```

means:

`this.accountId` = the class variable

and:

`accountId` = the constructor parameter

So:

```
this.accountId
      ↑
class variable

accountId
      ↑
parameter
```

23. The `execute()` method

apex

```
public void execute(QueueableContext context)
```

This is where Salesforce executes our background work.

Inside it:

apex

```
Account acc = [  
    SELECT Id, Description  
    FROM Account  
    WHERE Id = :accountId  
];
```

We query the Account.

Then:

apex

```
acc.Description = 'Processed by Queueable';
```

Change its Description.

Finally:

apex

```
update acc;
```

Save it to Salesforce.

24. How do we call this Queueable?

Suppose we have:

apex

```
Account acc;
```

Then:

apex

```
System.enqueueJob(  
    new AccountQueueableProcessor(acc.Id)  
);
```

This does **three things** conceptually:

```
acc.Id  
↓
```

```
new AccountQueueableProcessor(acc.Id)
↓
Create Queueable object
↓
System.enqueueJob()
↓
Put it in queue
```

Later:

```
execute()
```

runs.

25. Full Execute Anonymous Example

apex

```
Account acc = [
    SELECT Id
    FROM Account
    LIMIT 1
];

System.enqueueJob(
    new AccountQueueableProcessor(acc.Id)
);
```

26. Now write the Test Class

Here is the complete test:

apex

```
@isTest
public class AccountQueueableProcessorTest {

    @isTest
    static void testAccountQueueable() {

        Account acc = new Account(
            Name = 'Test Account'
        );

        insert acc;
```

```

Test.startTest();

System.enqueueJob(
    new AccountQueueableProcessor(acc.Id)
);

Test.stopTest();

Account updatedAcc = [
    SELECT Id, Description
    FROM Account
    WHERE Id = :acc.Id
];

System.assertEquals(
    'Processed by Queueable',
    updatedAcc.Description
);
}

```

Now let's go line by line.

27. Create test data

apex

```

Account acc = new Account(
    Name = 'Test Account'
);

```

Creates an Account in memory.

Then:

apex

```

insert acc;

```

actually creates it in Salesforce.

Now we have:

apex

```

acc.Id

```

28. Start the test

apex

```
Test.startTest();
```

Now we begin the asynchronous portion.

29. Enqueue the job

apex

```
System.enqueueJob(  
    new AccountQueueableProcessor(acc.Id)  
);
```

This is important.

We create our Queueable:

apex

```
new AccountQueueableProcessor(acc.Id)
```

and pass the Account Id.

Then:

apex

```
System.enqueueJob()
```

puts it into the queue.

30. Stop the test

apex

```
Test.stopTest();
```

This causes the queued job to execute during the test.

After this line, we can check the result.

31. Query the Account

```
apex

Account updatedAcc = [
    SELECT Id, Description
    FROM Account
    WHERE Id = :acc.Id
];
```

We're checking what happened to the Account after the Queueable ran.

32. Assert

```
apex

System.assertEquals(
    'Processed by Queueable',
    updatedAcc.Description
);
```

We're saying:

```
Expected:
Processed by Queueable

Actual:
updatedAcc.Description
```

If they match:

✅ Test passes.

Otherwise:

❌ Test fails.

33. Complete flow

You should be able to visualize this:

```

TEST
|
|— Create Account
|
|— Insert Account
|
|— Test.startTest()
|
|— new AccountQueueableProcessor(acc.Id)
|
|— System.enqueueJob()
|
|— Test.stopTest()
|
|   ↓
|   Queueable executes
|   ↓
|   Query Account
|   ↓
|   Change Description
|   ↓
|   Update Account
|
|— Query Account again
|
|— System.assertEquals()

```

34. Example Question 3 — Multiple Accounts

Now let's make the question slightly harder.

Create a Queueable Apex class that accepts a list of Account IDs and changes the Description of all those Accounts to "Processed" .

This is a very useful pattern.

Queueable

apex

```

public class AccountQueueableProcessor implements Queueable {

    private List<Id> accountIds;

    public AccountQueueableProcessor(List<Id> accountIds) {

```

```

        this.accountIds = accountIds;

    }

    public void execute(QueueableContext context) {

        List<Account> accounts = [
            SELECT Id, Description
            FROM Account
            WHERE Id IN :accountIds
        ];

        for (Account acc : accounts) {

            acc.Description = 'Processed';

        }

        update accounts;

    }
}

```

35. Understand the important difference

Previously:

apex

```
private Id accountId;
```

Now:

apex

```
private List<Id> accountIds;
```

We're storing multiple IDs.

Then constructor:

apex

```
public AccountQueueableProcessor(List<Id> accountIds)
```

accepts the list.

36. Query multiple records

apex

```
List<Account> accounts = [  
    SELECT Id, Description  
    FROM Account  
    WHERE Id IN :accountIds  
];
```

This gets all Accounts whose IDs are in our list.

37. Loop

apex

```
for (Account acc : accounts) {  
  
    acc.Description = 'Processed';  
  
}
```

Change every Account.

38. One DML statement

apex

```
update accounts;
```

Very important:

Don't do:

apex

```
for (Account acc : accounts) {  
    update acc;  
}
```

Do:

apex

```
for (Account acc : accounts) {
    acc.Description = 'Processed';
}

update accounts;
```

That's **bulkification**.

39. Test class

apex

```
@isTest
public class AccountQueueableProcessorTest {

    @isTest
    static void testMultipleAccounts() {

        List<Account> accounts = new List<Account>();

        for (Integer i = 0; i < 3; i++) {

            accounts.add(
                new Account(
                    Name = 'Test Account ' + i
                )
            );
        }

        insert accounts;

        List<Id> accountIds = new List<Id>();

        for (Account acc : accounts) {

            accountIds.add(acc.Id);

        }

        Test.startTest();

        System.enqueueJob(
            new AccountQueueableProcessor(accountIds)
        );

        Test.stopTest();

        List<Account> updatedAccounts = [
            SELECT Id, Description
```

```

        FROM Account
        WHERE Id IN :accountIds
    ];

    for (Account acc : updatedAccounts) {

        System.assertEquals(
            'Processed',
            acc.Description
        );
    }
}

```

40. What's happening here?

We create:

```

Account 1
Account 2
Account 3

```

Then:

```
[Account1.Id, Account2.Id, Account3.Id]
```

gets passed to:

```

apex

new AccountQueueableProcessor(accountIds)

```

Then Queueable processes all three.

★ 41. Queueable Chaining

This is one of the BIG reasons Queueable is more powerful than Future.

Suppose we have:

```

Queueable 1
    ↓
Queueable 2

```

A Queueable can enqueue another Queueable.

Example Question 4

Create a Queueable class that updates an Account and then starts another Queueable to perform another operation.

First Queueable:

```
apex

public class FirstQueueable implements Queueable {

    private Id accountId;

    public FirstQueueable(Id accountId) {
        this.accountId = accountId;
    }

    public void execute(QueueableContext context) {

        Account acc = [
            SELECT Id, Description
            FROM Account
            WHERE Id = :accountId
        ];

        acc.Description = 'Step 1 completed';

        update acc;

        System.enqueueJob(
            new SecondQueueable(accountId)
        );
    }
}
```

Notice this:

```
apex

System.enqueueJob(
    new SecondQueueable(accountId)
);
```

That's the chaining.

Second Queueable

apex

```
public class SecondQueueable implements Queueable {

    private Id accountId;

    public SecondQueueable(Id accountId) {
        this.accountId = accountId;
    }

    public void execute(QueueableContext context) {

        Account acc = [
            SELECT Id, Description
            FROM Account
            WHERE Id = :accountId
        ];

        acc.Description = 'Step 2 completed';

        update acc;
    }
}
```

Flow:

```
FirstQueueable
    ↓
Update Account
    ↓
enqueue SecondQueueable
    ↓
SecondQueueable
    ↓
Update Account
```

This kind of sequential asynchronous processing is one reason Queueable is useful for multi-step jobs.

42. Queueable Keywords Cheat Sheet

Here's what you should memorize.

Code	Meaning
<code>public</code>	Accessible class/method
<code>class</code>	Creates a class
<code>implements Queueable</code>	Makes class a Queueable job
<code>private</code>	Variable is used within class
<code>Id</code>	Salesforce record ID
<code>List<Id></code>	Multiple record IDs
Constructor	Pass data into Queueable
<code>this</code>	Refers to current object's variable
<code>execute()</code>	Where Queueable logic runs
<code>QueueableContext</code>	Context of Queueable execution
<code>void</code>	No return value
<code>System.enqueueJob()</code>	Puts Queueable into async queue
<code>Test.startTest()</code>	Starts test execution context
<code>Test.stopTest()</code>	Executes async work during test
<code>System.assertEquals()</code>	Verifies expected vs actual result

43. Queueable Template — Memorize This

When you're given a coding question, start with:

apex

```
public class MyQueueable implements Queueable {

    private Id recordId;

    public MyQueueable(Id recordId) {

        this.recordId = recordId;
```

```

    }

    public void execute(QueueableContext context) {

        // Query

        // Logic

        // DML

    }
}

```

Then call:

```

apex

System.enqueueJob(
    new MyQueueable(recordId)
);

```

44. Queueable Test Template

Memorize this structure:

```

apex

@isTest
public class MyQueueableTest {

    @isTest
    static void testQueueable() {

        // Create data
        Account acc = new Account(
            Name = 'Test Account'
        );

        // Insert data
        insert acc;

        // Start async test
        Test.startTest();

        // Enqueue Queueable
        System.enqueueJob(
            new MyQueueable(acc.Id)
        );
    }
}

```

```

    );

    // Execute async job
    Test.stopTest();

    // Query result
    Account result = [
        SELECT Id, Description
        FROM Account
        WHERE Id = :acc.Id
    ];

    // Verify result
    System.assertEquals(
        'Expected Value',
        result.Description
    );
}
}

```

45. The most important difference from Future

This is what I want you to remember from this lesson.

Future:

apex

```

@future
public static void process(Id id) {

}

```

Call:

apex

```

MyClass.process(id);

```

Queueable:

apex

```

public class MyClass implements Queueable {

    private Id id;
}

```

```

    public MyClass(Id id) {
        this.id = id;
    }

    public void execute(QueueableContext context) {

    }
}

```

Call:

```

    apex

    System.enqueueJob(
        new MyClass(id)
    );

```

So when you see:

"Create a Queueable class..."

Your brain should immediately go:

```

implements Queueable
    ↓
private variables
    ↓
constructor
    ↓
execute(QueueableContext context)
    ↓
SOQL
    ↓
Logic
    ↓
DML
    ↓
System.enqueueJob(new ClassName(...))

```

And for the test:

```

Create test data
    ↓
insert
    ↓
Test.startTest()
    ↓
System.enqueueJob()

```

```
↓  
Test.stopTest()  
  
↓  
query result  
  
↓  
System.assertEquals()
```

Next, Batch Apex is where the syntax changes significantly: `Database.Batchable`, `start()`, `execute()`, `finish()`, `Database.QueryLocator`, batch size, and its test class.

That's the one to learn carefully because you'll almost certainly encounter coding questions around all three methods.